



抖音Android启动优化实践

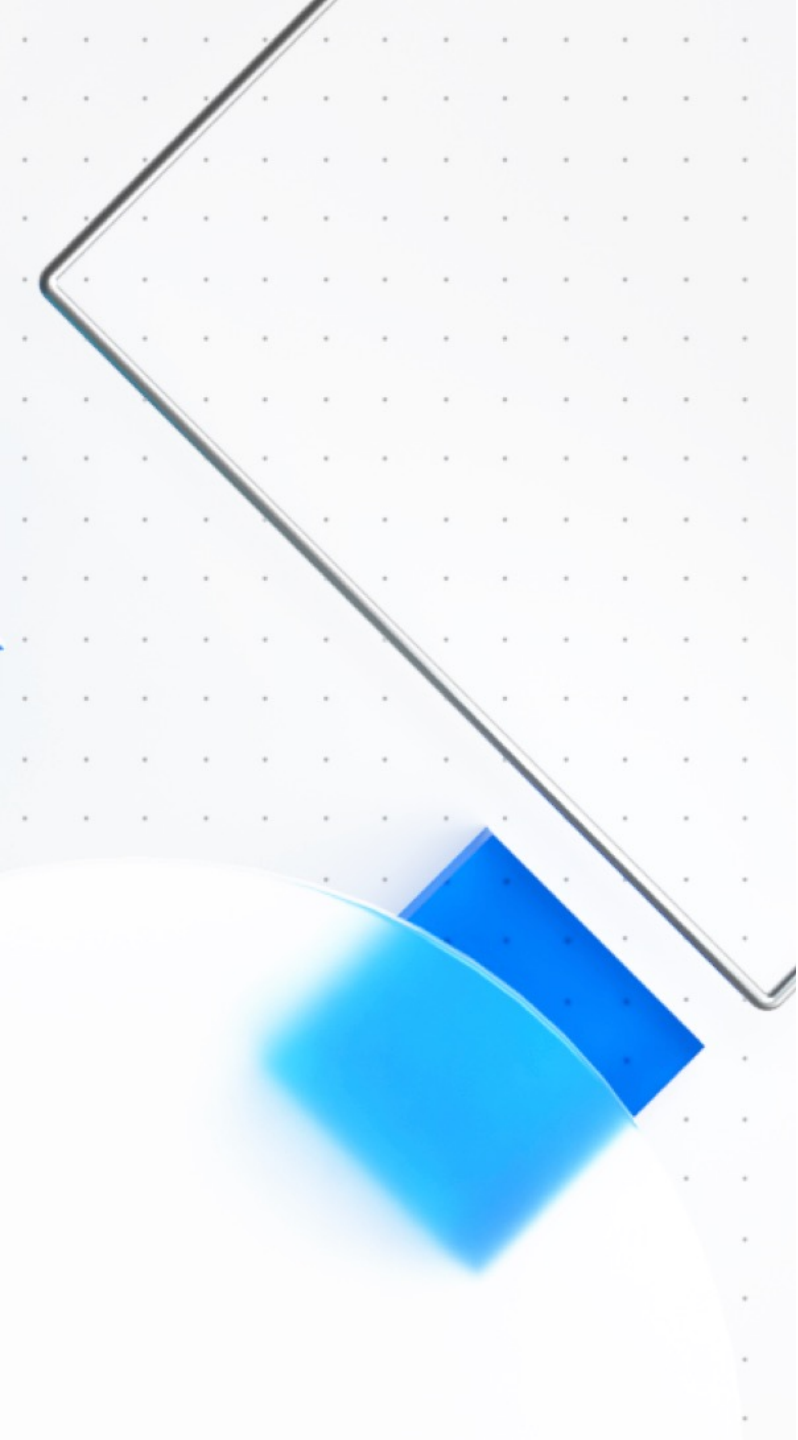
冯达

自我介绍



冯达

毕业于浙江大学，2014年开始从事 Android 相关的研发工作，先后在猎豹移动、360就职，期间有过应用开发、插件化、性能优化等方面的工作经验。2019年加入字节跳动，主要从事体验优化相关工作，目前负责抖音的启动以及业务相关的体验优化。



01 抖音启动优化开展思路

02 优化案例解析

03 总结与展望

01

抖音启动优化开展思路

启动优化开展思路

01

为什么优化

希望通过优化达成什么目标

02

优化什么

有哪些优化方向，应该优化哪些内容

03

如何优化

通过哪些的优化手段达成优化目标

01_为什么进行启动优化?

启动速度太慢无法忍受?

达成秒开的技术目标?

超越所有竞品达到业界第一?

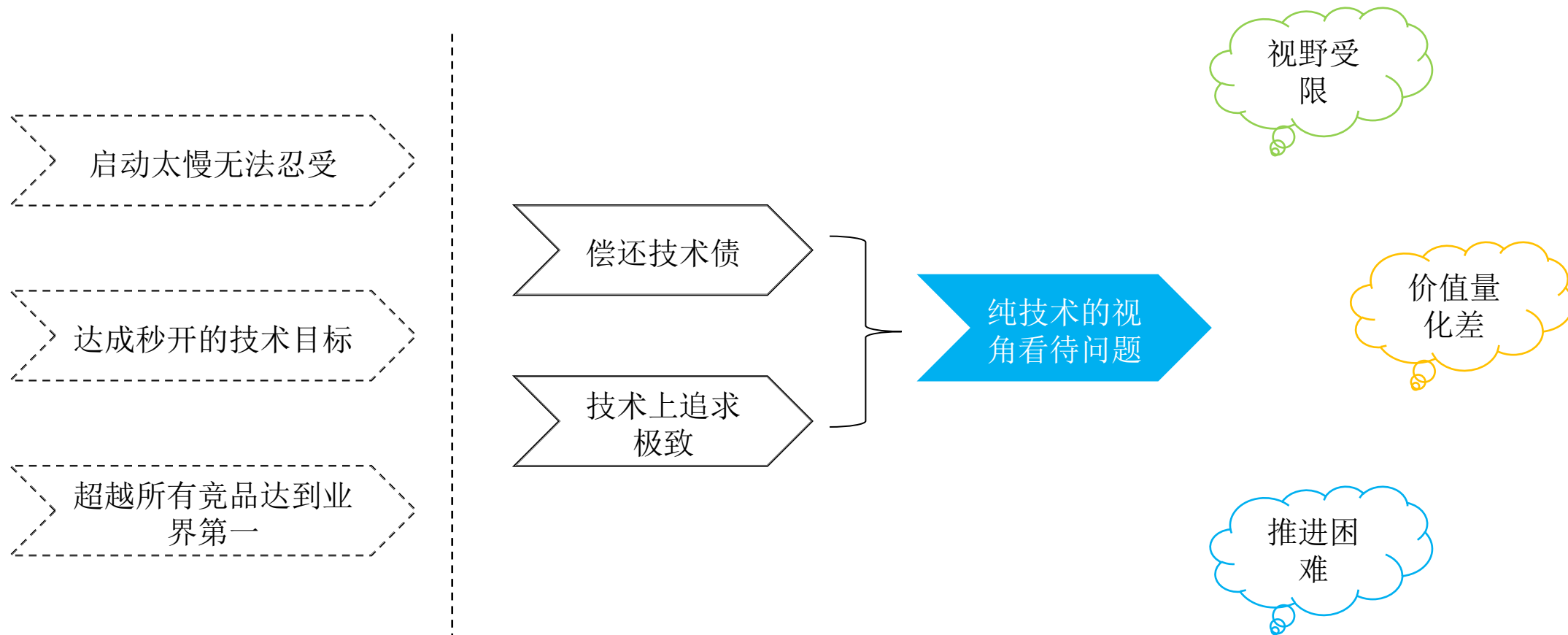
需要优化到什么程度?

秒开的必要性? 秒开是否就足够了?

竞品也进行优化怎么办?

竞品业务逻辑非常简单怎么办?

01_为什么进行启动优化



01_为什么进行启动优化

用做**业务**的思维做体验优化

场景：各类启动场景

指标：留存、使用时长、广告收入等全局指标

以及各个业务的子指标

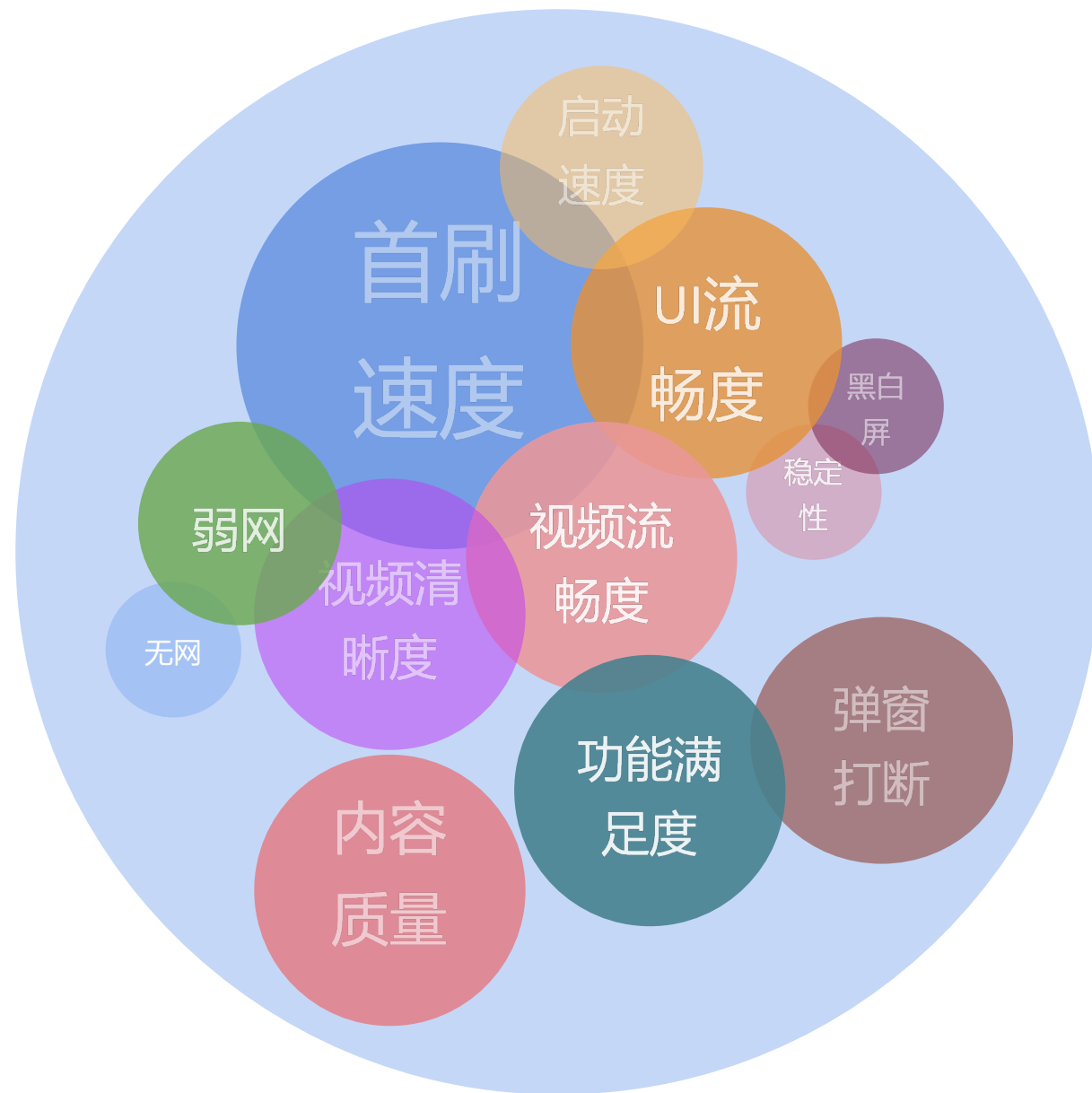
目标：**通过启动优化促进业务指标提升**



02_优化什么

启动场景所有能带来体验提升的优化

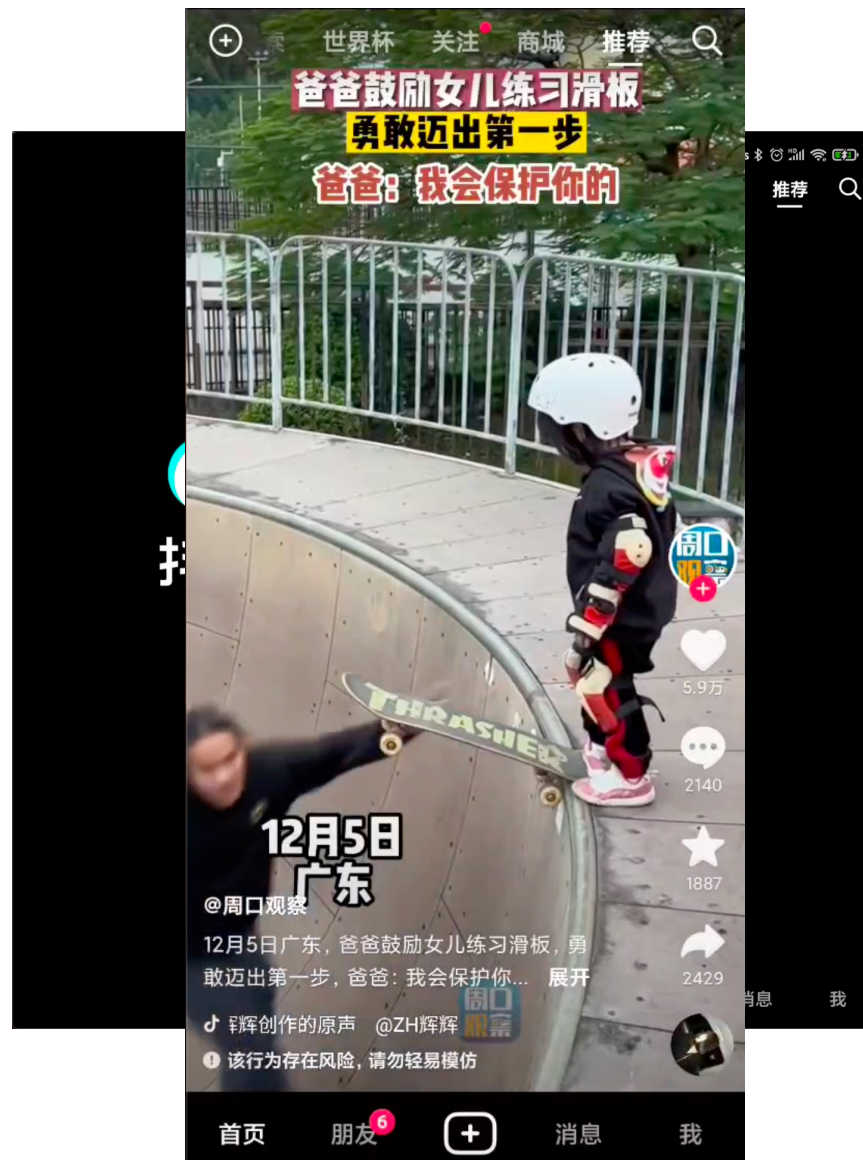
- **场景泛化:**启动完成不意味着启动场景的结束，启动开始后的一段时间（10s、20s、60s）都属于启动场景；
- **优化内容泛化:**启动体验不只有启动速度，内容加载速度以及内容加载完成后一段时间的性能、功能等都是启动优化所需要优化的



02_优化什么

优化内容拆解

- **功能的触达速度：** 启动速度、首刷速度
- **功能的品质**
 - 功能满足度：该有的功能是否有
 - 已有功能品质是否足够高
 - 内容质量：个性化、视频本身质量、时效性等
 - 显示质量：视频清晰度、内容显示完整性等
 - 流畅性：弹窗打断、UI卡顿、视频缓冲等
 - 特殊场景：弱网、黑白屏问题、稳定性



03_如何进行优化



03_如何进行优化

价值论证

- 理论分析：从原理出发分析可能的优化空间
- 价值预估
 - 历史优化论证：通过历史优化，预估未来优化的收益空间
 - 劣化实验：对某些指标进行劣化，预估未来优化空间
- 定期review：避免预估失准、边际效应等问题



03_如何进行优化

问题发现

线下问题发现

- 日志、trace、代码分析
- 竞品测试、体验走查
- 体验产品

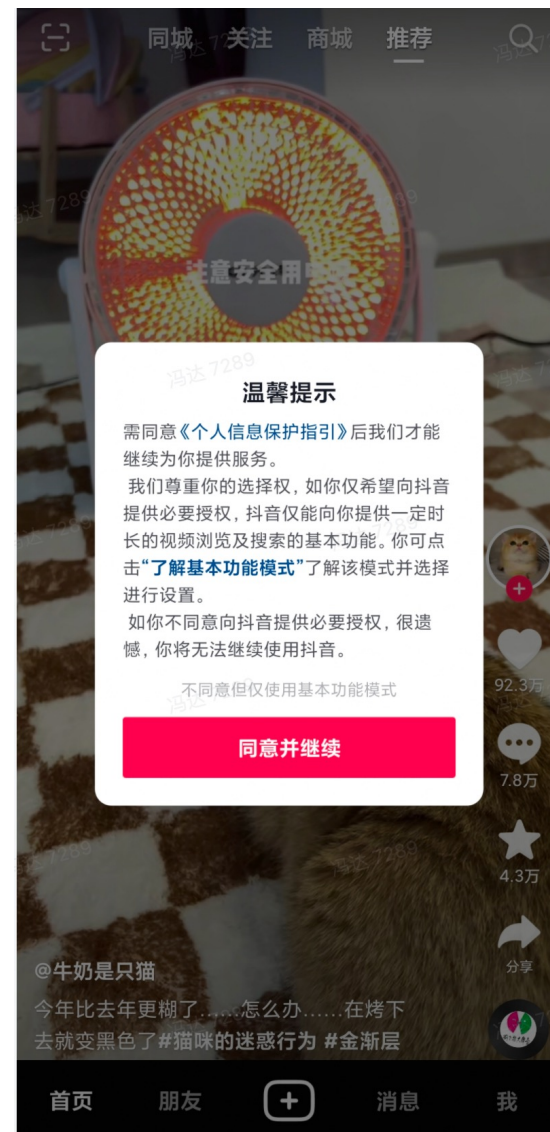
线上问题发现

- 数据监控、APM工具
- 用户调研、用户反馈

03_如何进行优化

问题优化：功能优化

- 原则：挖掘用户诉求，以产品、技术视角尽可能去满足用户诉求
- 案例1：基本模式
 - 背景
 - 用户无法在不同意隐私弹窗情况下使用应用
 - 每天有xxx万次不同意行为上报
 - 用户诉求：在不同意隐私弹窗情况下使用应用
 - 解决办法：
 - 未同意隐私弹窗情况下不收集任何信息
 - 不进行个性化推荐，仅推荐高热视频
- 案例2:内容推荐优化（见案例解析）



03_如何进行优化

问题优化：性能分析

- 全链路分析：

- 对整个主链路进行完整分析；
- 主链路以外的间接影响分析；



发现容易被忽略的问题

- 耗时成因分析

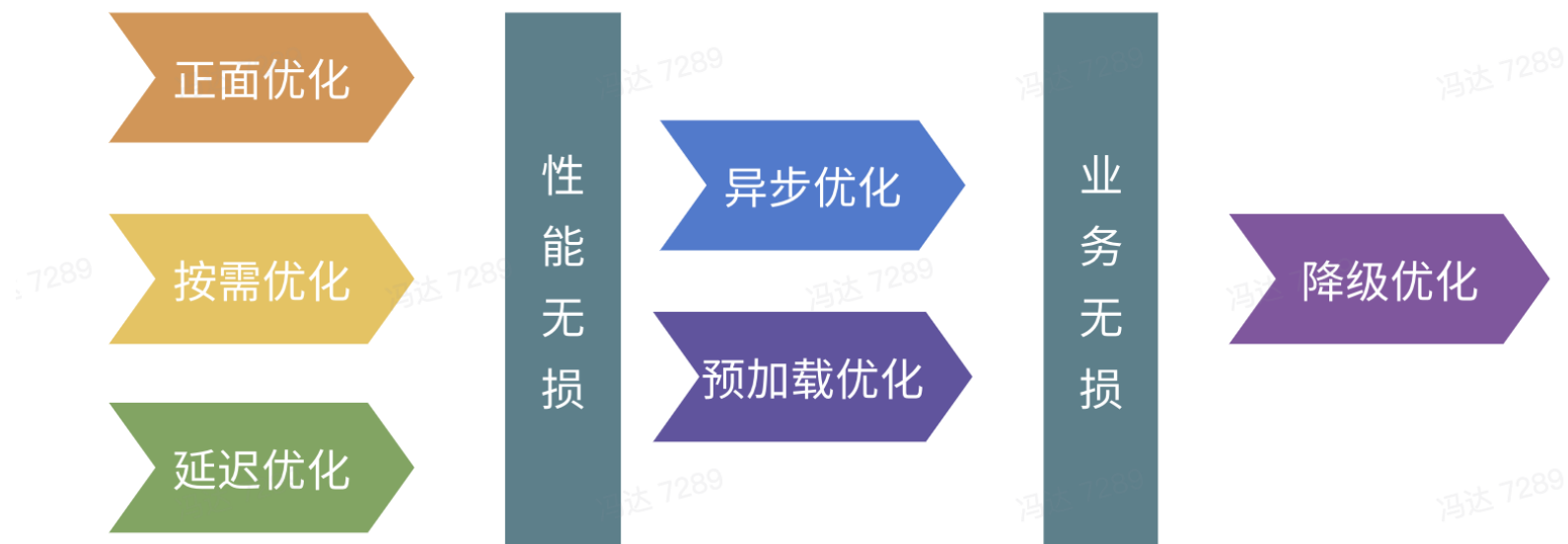
- CPU Time：循环、序列化反序列化、类解析、反射等
- IO： IO操作，等到IO结果返回
- 锁等待：等锁状态，等待其他线程释放锁、唤醒或者自己超时唤醒
- Runnale：资源抢占导致分配不到cpu时间片



更极致、创新的优化策略

03_如何进行优化

问题优化：性能优化



03_如何进行优化

抖音启动性能优化

覆盖率最大化

关注所有启动场景，避免非核心启动场景被忽视，出现启动体验死角

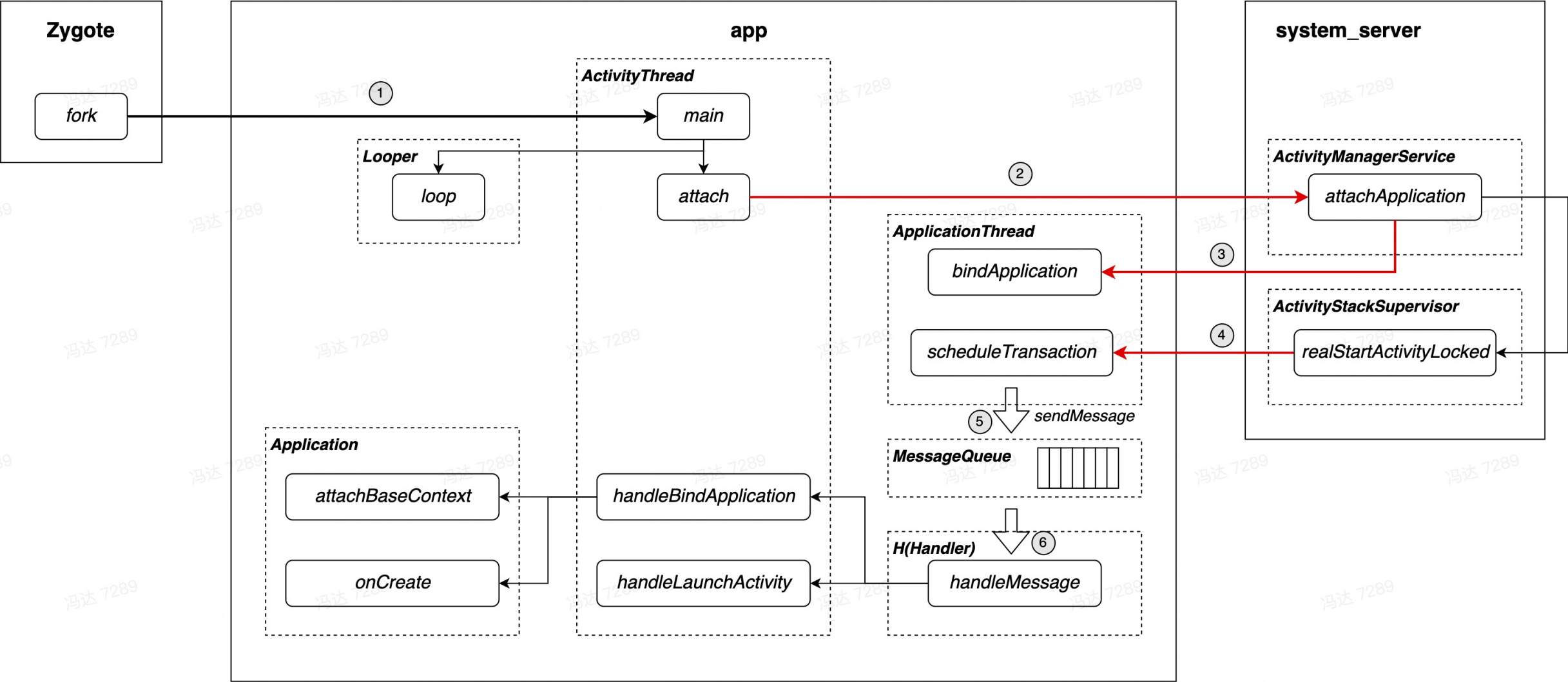
代码价值最大化

通过降级等方式，减少启动阶段无价值任务的执行，提升代码价值

资源利用率最大化

在启动任务一定的情况下，通过技术优化以更合理、充分地利用系统资源

03_如何进行优化



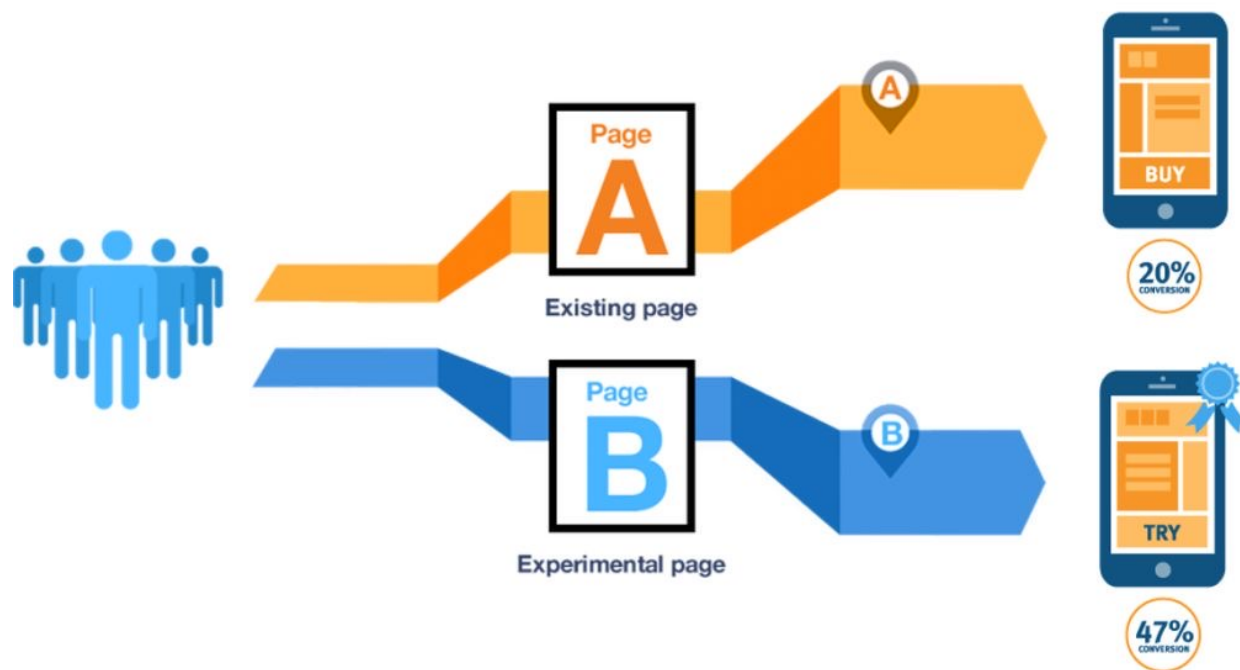
03_如何进行优化

资源利用率最大化

- 特点：全局最优、业务无损
- 落地路径：
 - 正面优化：对单个任务通过优化其实现，以减少资源消耗，以提供更多资源给其他任务
 - 业务实现的优化
 - 基础框架、framework、虚拟机等通用优化
 - 调度优化：对多个任务通过优化任务排布，更充分与合理的利用资源
 - 任务优先级、任务资源类型(cpu、io)、依赖关系等；
 - 并发度控制、线程优先级
- 案例：Gson框架优化、线程收敛、类加载优化（见案例解析部分）

03_如何进行优化

线上验证：AB实验（火山引擎）



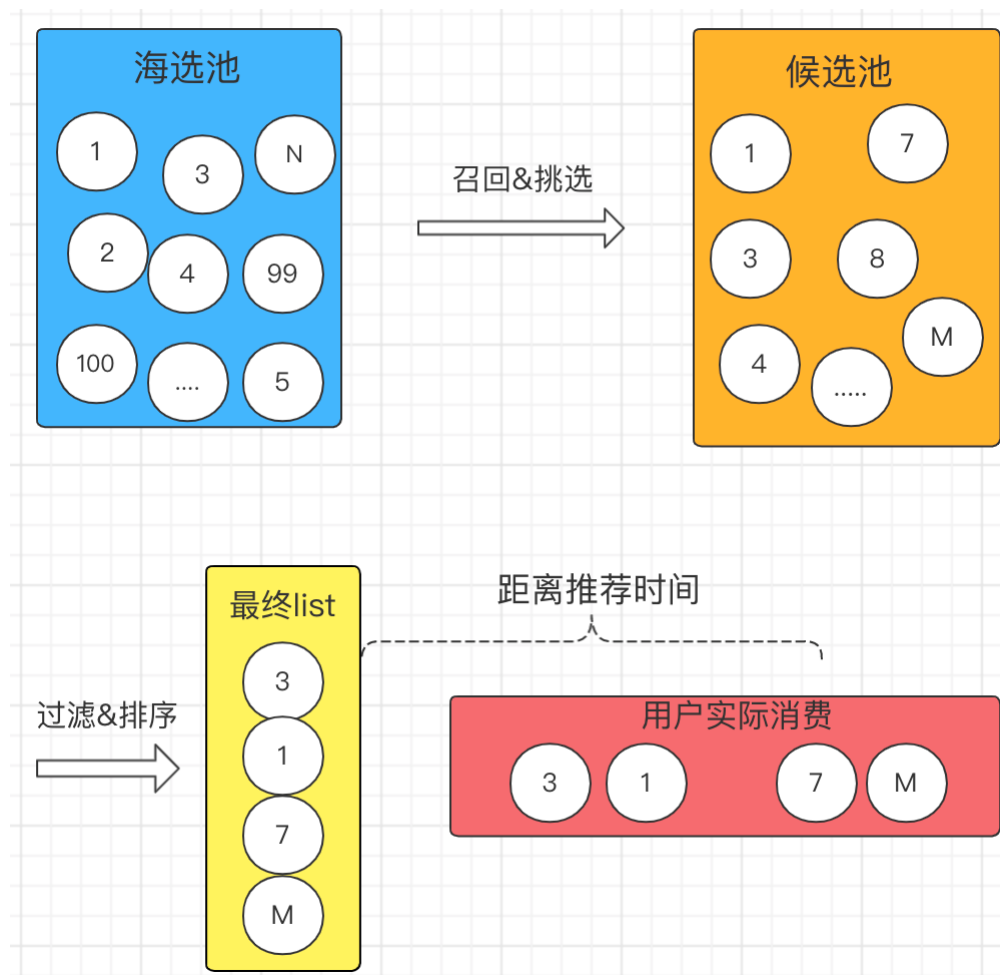
02

优化案例解析

案例1_内容推荐优化

• 推荐链路分析

- **挑选：** 从视频海选池（亿级）中根据用户画像等信息挑选出候选的视频（数百条）
- **排序与过滤：** 对候选池子中的视频进行过滤与排序，并挑选出排名靠前的k条视频作为最终的返回结果
- **用户消费：** 推荐结果返回给用户，用户在延后xx秒开始逐条进行消费

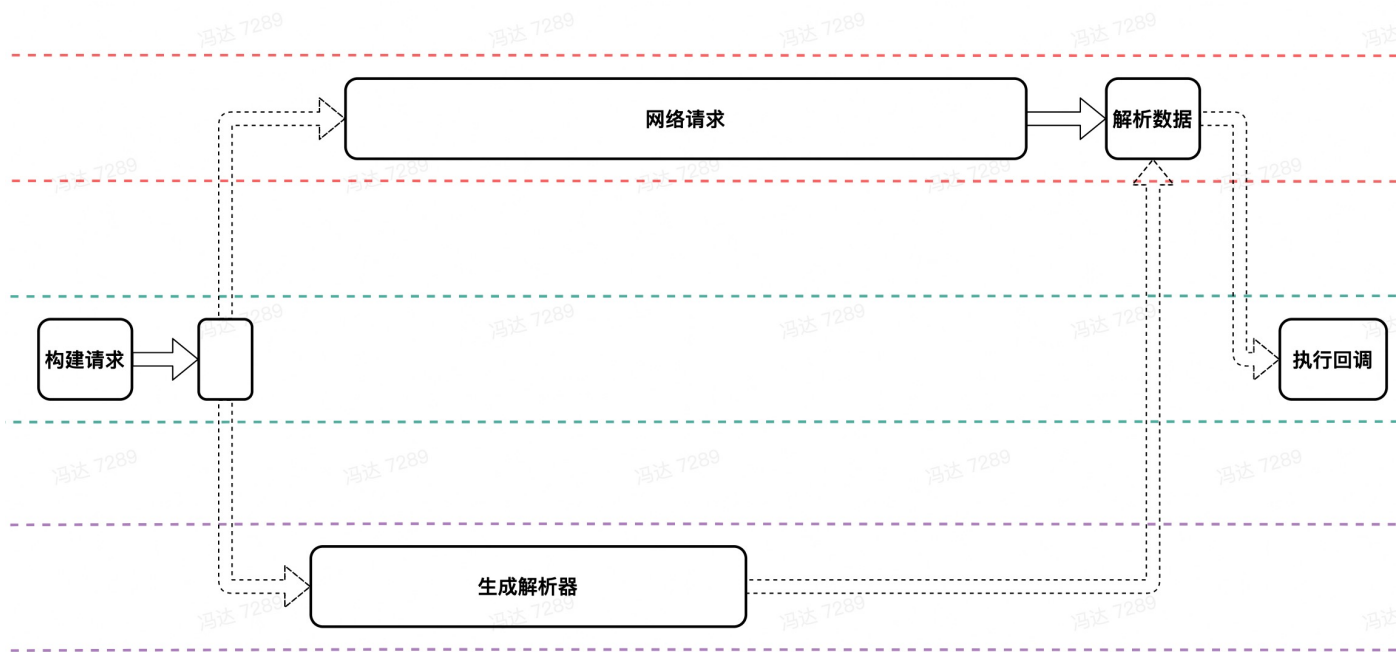


案例1_内容推荐优化

- **挑选阶段：** 从视频海选池（亿级）中根据**用户画像**等信息挑选出候选的视频（数百条）
 - **更多画像画像：** 基础画像、实时画像
 - **更快画像同步：** 打点提频、新通道、关键信息变更即时反馈
- **排序与过滤阶段：** 依照给定的**策略**对候选池子中的视频进行过滤与排序，并挑选出排名靠前的k条视频
 - **差异化的排序与过滤策略**
- **消费阶段：** 推荐结果返回给用户，用户在**延后xx秒**开始进行消费并给予**反馈**
 - **时效性优化：** 过期数据替换、时效性增强
 - **用户反馈优化：** 连续skip、like、dislike等反馈

案例2_Gson框架优化

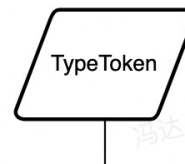
- **背景：**当class结构复杂时首次进行数据解析非常耗时
- **问题原因：**Gson基于反射实现，当class结构复杂存在较多嵌套时，解析class会极为耗时；
- **优化方案**
 - 避免解析：快照
 - 同步改异步：注入自定义ConverterFactory将同步改为异步，优化Retrofit性能



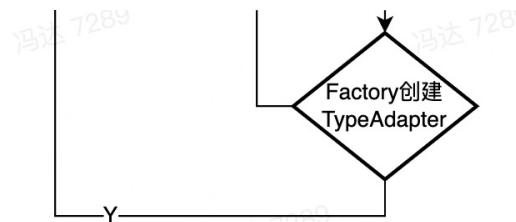
案例2_Gson框架优化

- 缓存优化

- 预加载：通过getAdapter方法预解析
- 统一Gson对象：
 - 使用统一接口获取Gson对象
 - 自动统一默认Gson对象
- 多线程优化：将缓存粒度从class维度细化为field维度
- 跨Gson对象优化：field信息从Gson对象缓存变成全局缓存（lru cache）



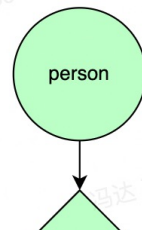
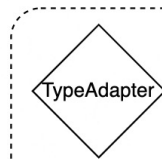
```
Gson gson = new Gson();  
for (int i = 0; i < 100; i++) {  
    gson.fromJson(JSON, JavaBean.class);  
}  
  
for (int i = 0; i < 100; i++) {  
    new Gson().fromJson(JSON, JavaBean.class);  
}
```



案例2_Gson框架优化

- 反射优化

- 按需解析：根据实际数据解析、避开预加载场景
- 空间换时间：编译期生成解析代码
- 其他解析协议：PB、其他二进制序列化协议（Android的Parcel）

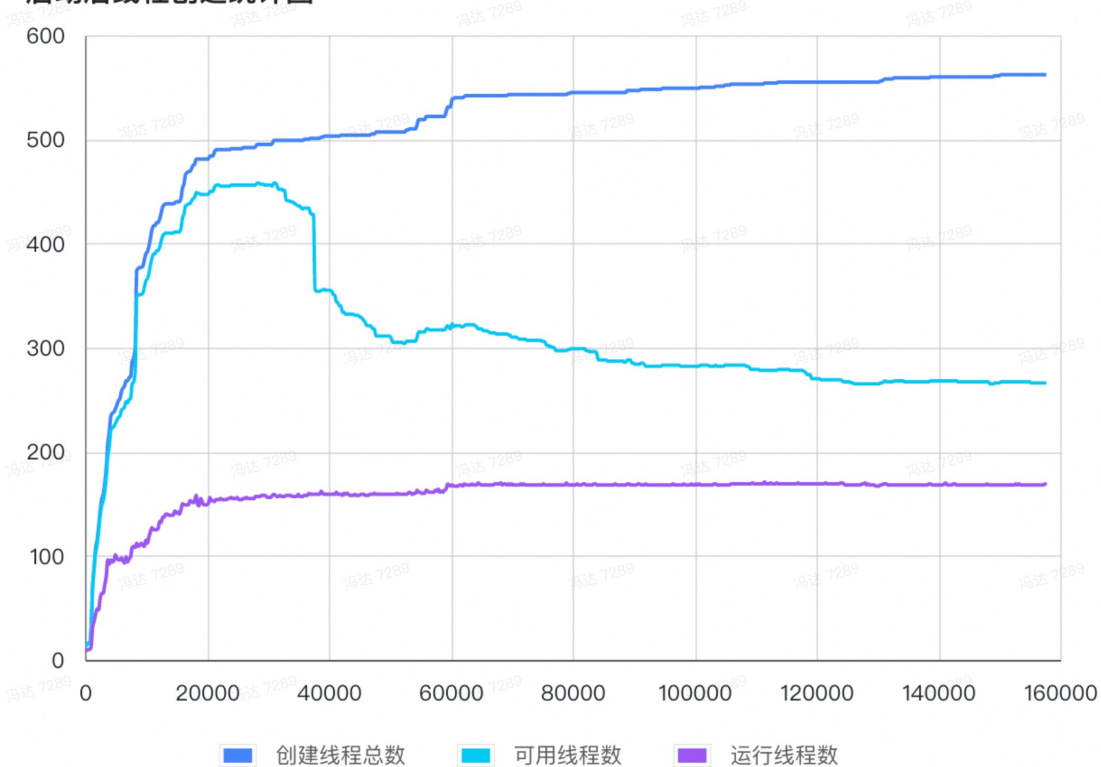


```
public abstract class TypeAdapterForModel1 extends BaseTypeAdapter {  
  
    protected void setFieldValue(String var1, Object var2, JsonReader var3) {  
        Object var4;  
        switch(var1.hashCode()) {  
            case 110371416:  
                if (var1.equals("field1")) {  
                    var4 = this.gson.getAdapter(String.class).read(var3);  
                    ((Model1)var2).field1 = (String)var4;  
                    return true;  
                }  
                break;  
            case 1223751172:  
                if (var1.equals("field2")) {  
                    var4 = this.gson.getAdapter(String.class).read(var3);  
                    ((Model1)var2).field2 = (String)var4;  
                    return true;  
                }  
            }  
        }  
        return false;  
    }  
}
```

案例3_线程收敛优化

- **问题背景：**大量线程并发抢占资源影响启动性能、大量线程创建相关oom；
- **问题原因：**各业务、中台、三方sdk为了实现内部闭环，都有自己的线程池，缺乏整体管控
- **优化方向：**优化线程并发、减少闲置线程数量、提升线程复用率

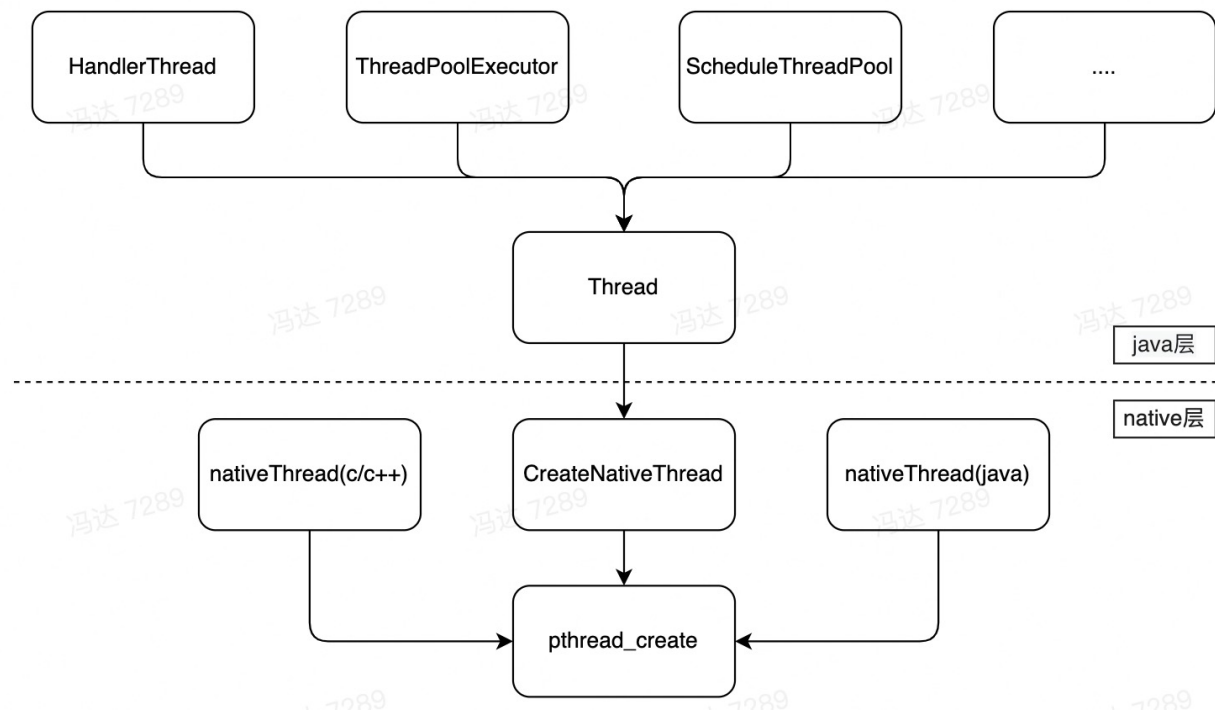
启动后线程创建统计图



案例3_线程收敛优化

- 优化方案

- 业务改造：提供统一的线程池，推进业务进行接入
- 超级线程池：自动接管线程的创建，所有线程都从超级线程池获取；
 - Java层方案：字节码处理，兼容性好；
 - Native层方案：运行时hook，覆盖面更大；
 - 多级膨胀与收缩机制、异常监控、白名单等；



案例4_类加载优化

- 背景知识

- 双亲委派机制
- Android的ClassLoader: BootClassLoader、PathClassLoader

- 优化思路: 从原理出发, 找出优化点

- 类加载过程: load、link (verify、prepare、resolve)

```
protected Class<?> loadClass(String name, boolean resolve)
    throws ClassNotFoundException
{
    Class<?> c = findLoadedClass(name);
    if (c == null) {
        try {
            if (parent != null) {
                c = parent.loadClass(name, false);
            } else {
                c = findBootstrapClassOrNull(name);
            }
        } catch (ClassNotFoundException e) {
        }

        if (c == null) {
            c = findClass(name);
        }
    }
    return c;
}
```

案例4_类加载优化

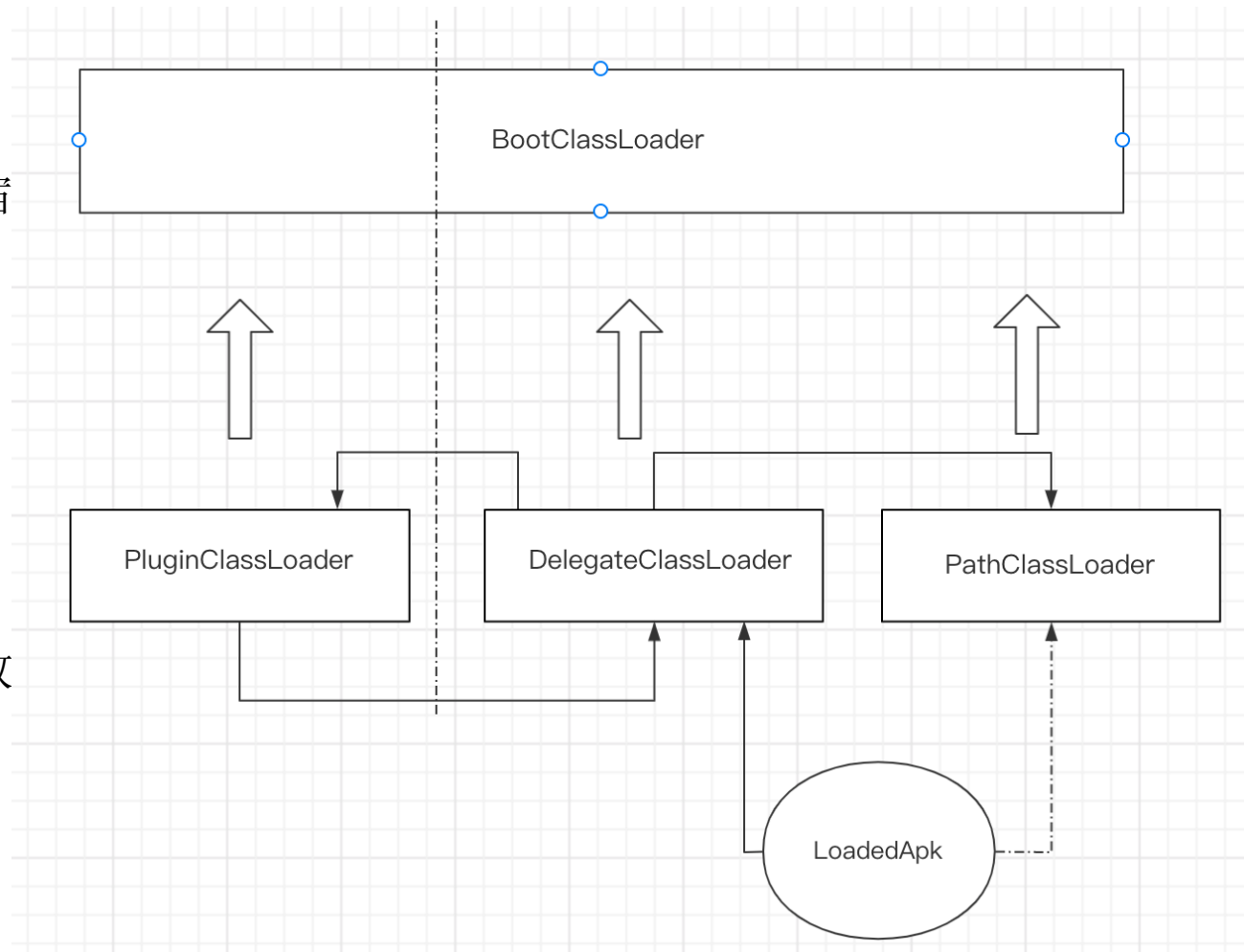
- load优化

- 抖音的ClassLoader模型

- 优点：插件与宿主类隔离、插件复用宿主的class、业务无感加载插件类
 - 缺点：破坏了ART类加载优化

- 解决办法

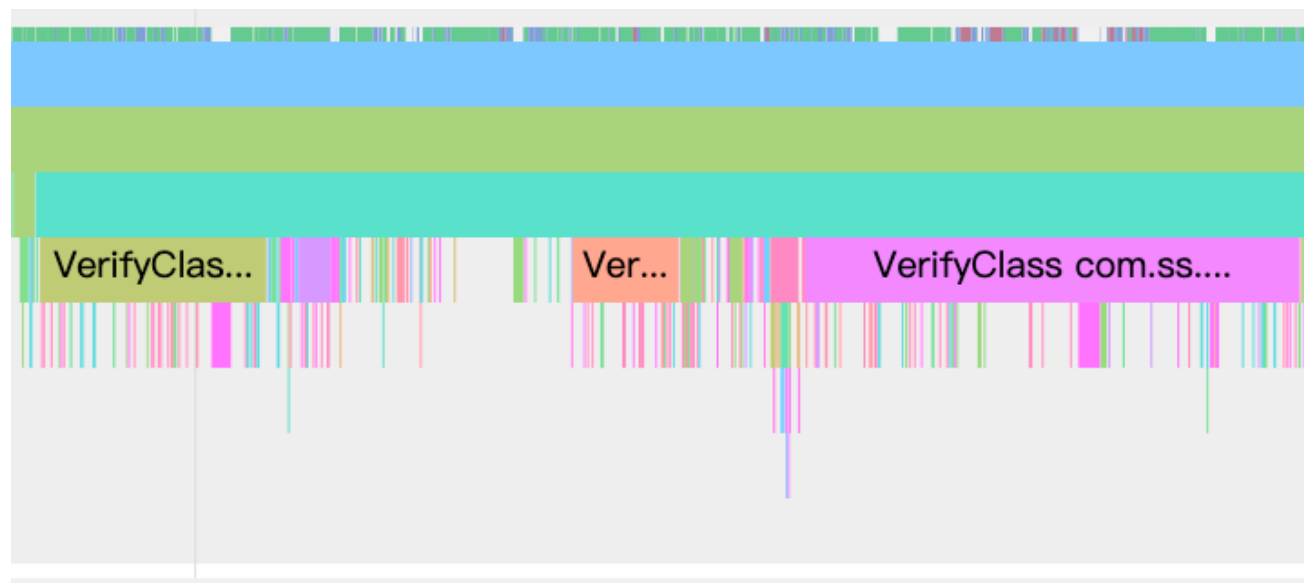
- 修改ClassLoader模型
 - 对反射代码进行插桩，感知类加载失败
 - 改造CompileOnly依赖方式
 - 替换framework的ClassLoader



案例4_类加载优化

- verify优化

- 原因：安装时verify失败、android10以后的的插件class
- 不必要性：
 - 编译期已经进行了字节码校验，无需运行时verify
 - 即使有不合法的字节码，是否进行verify无影响
- 方案：修改runtime.h中verify_字段内存值实现disable verify



案例4_类加载优化

- dex2oat优化

- Jit + AOT混合编译:dexiamege、热方法机器码

- 主动触发: cmd package compile -m speed-profile -f xxxxx

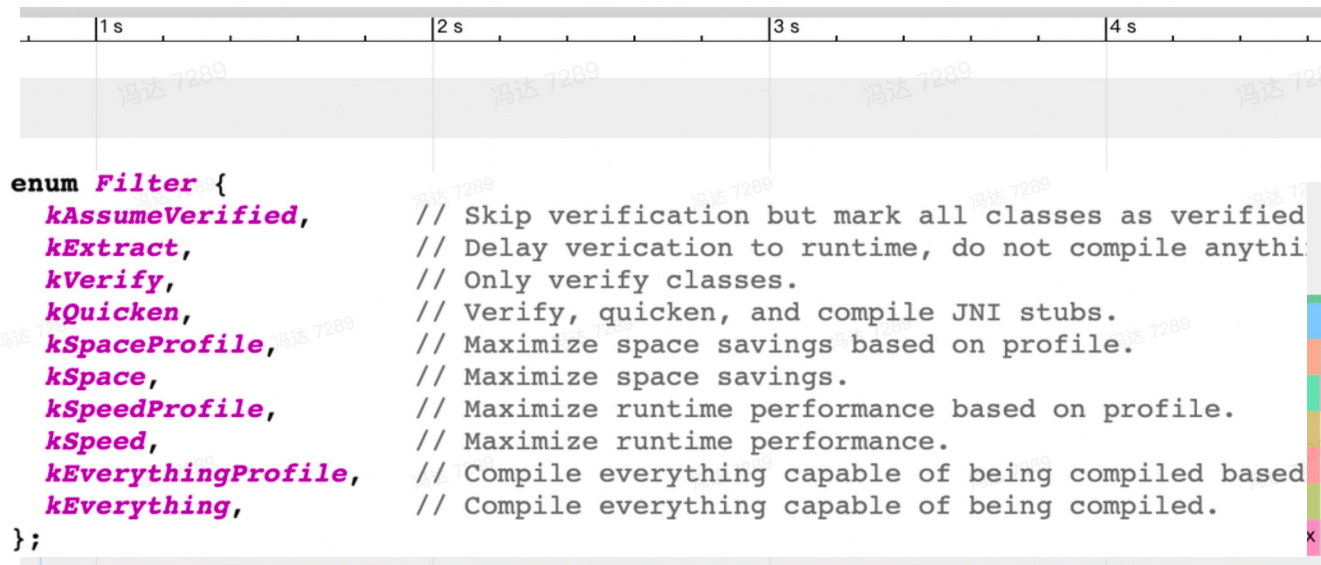
- dex2oat失败:

- 失败检测: shell命令返回值、
openDexFilesFromOat耗时

- 失败处理: 重试、降级

- BaselineProfile: 内置profile文件, 安装进行预编译;

- ProfileInstaller: 运行时写入profile信息;



03

总结与展望

03

- 总结
 - 用做业务的思维做启动优化
 - 优化贯穿应用层、通用框架、framework、虚拟机
- 展望
 - 体验优化与架构优化并存，降低启动复杂度
 - 攻防兼备，优化、管控、防劣化并存



THANKS.